

1. Implement a Stack using Array data structure.

Program

```
#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push(int x) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = x;
}

void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else
        printf("Popped: %d\n", stack[top--]);
}

void display() {
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack: ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

Output

```
Stack: 30 20 10
Popped: 30
Stack: 20 10
```

2. Implement a Queue using Stack data structure.

Program

```
#include <stdio.h>
#define MAX 5

int s1[MAX], s2[MAX], top1 = -1, top2 = -1;

void push1(int x) { s1[++top1] = x; }
int pop1() { return s1[top1--]; }
void push2(int x) { s2[++top2] = x; }
int pop2() { return s2[top2--]; }

void enqueue(int x) {
    if (top1 == MAX - 1)
        printf("Queue Overflow\n");
    else
        push1(x);
}

int dequeue() {
    int x;
    if (top1 == -1)
        return -1;

    while (top1 != -1)
        push2(pop1());

    x = pop2();

    while (top2 != -1)
        push1(pop2());

    return x;
}

void display() {
    int i;
    printf("Queue: ");
    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    printf("Deleted: %d\n", dequeue());
    display();
    return 0;
}
```

Output

```
Queue: 10 20 30
Deleted: 10
Queue: 20 30
```

3. Write a C program using stack data structure to manage the history of web pages visited.

Program

```
#include <stdio.h>
#include <string.h>
#define MAX 10

char back[MAX][50], forward[MAX][50];
int topBack = -1, topForward = -1;
char current[50] = "Home";

void visit(char page[]) {
    if (topBack < MAX - 1)
        strcpy(back[++topBack], current);

    strcpy(current, page);
    topForward = -1;
}

void goBack() {
    if (topBack == -1) {
        printf("No previous page\n");
        return;
    }
    strcpy(forward[++topForward], current);
    strcpy(current, back[topBack--]);
}

void goForward() {
    if (topForward == -1) {
        printf("No forward page\n");
        return;
    }
    strcpy(back[++topBack], current);
    strcpy(current, forward[topForward--]);
}

int main() {
    visit("Google");
    visit("YouTube");
    visit("GitHub");

    printf("Current page: %s\n", current);
    goBack();
    printf("After Back: %s\n", current);
    goBack();
    printf("After Back: %s\n", current);
    goForward();
    printf("After Forward: %s\n", current);

    return 0;
}
```

Output

```
Current page: GitHub
After Back: YouTube
After Back: Google
After Forward: YouTube
```

4. Write a C program to sort a stack using a temporary stack.

Program

```
#include <stdio.h>
#define MAX 10

int stack[MAX], top = -1;
int temp[MAX], ttop = -1;

void push(int s[], int *top, int x) {
    s[++(*top)] = x;
}

int pop(int s[], int *top) {
    return s[(*top)--];
}

void sortStack() {
    while (top != -1) {
        int x = pop(stack, &top);

        while (ttop != -1 && temp[ttop] > x)
            push(stack, &top, pop(temp, &ttop));

        push(temp, &ttop, x);
    }

    while (ttop != -1)
        push(stack, &top, pop(temp, &ttop));
}

void display() {
    int i;
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    push(stack, &top, 34);
    push(stack, &top, 3);
    push(stack, &top, 31);
    push(stack, &top, 98);
    push(stack, &top, 92);
    push(stack, &top, 23);

    printf("Before sorting: ");
    display();

    sortStack();

    printf("After sorting: ");
    display();

    return 0;
}
```

Output

```
Before sorting: 23 92 98 31 3 34
After sorting: 3 23 31 34 92 98
```

5. The Stock Span problem: Given daily stock prices, calculate the span for each day using a stack.

Program

```
#include <stdio.h>
#define MAX 20

int main() {
    int price[] = {100, 80, 60, 70, 60, 75, 85};
    int span[MAX], stack[MAX];
    int top = -1, n = 7, i;

    for (i = 0; i < n; i++) {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;

        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }

    printf("Stock prices: ");
    for (i = 0; i < n; i++)
        printf("%d ", price[i]);

    printf("\nSpan: ");
    for (i = 0; i < n; i++)
        printf("%d ", span[i]);

    return 0;
}
```

Output

```
Stock prices: 100 80 60 70 60 75 85
Span: 1 1 1 2 1 4 6
```

6. Given an array, for each element find the nearest element to the right that has a higher frequency than the current element. If no such element exists, return -1. Implement using stack.

Program

```
#include <stdio.h>
#define MAX 20

int main() {
    int a[] = {2, 1, 1, 3, 2, 1};
    int n = 6, freq[MAX] = {0};
    int result[MAX], stack[MAX], top = -1;
    int i, j;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (a[i] == a[j])
                freq[i]++;

    for (i = n - 1; i >= 0; i--) {
        while (top != -1 && freq[stack[top]] <= freq[i])
            top--;

        if (top == -1)
            result[i] = -1;
        else
            result[i] = a[stack[top]];

        stack[++top] = i;
    }

    printf("Input: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nOutput: ");
    for (i = 0; i < n; i++)
        printf("%d ", result[i]);

    return 0;
}
```

Output

```
Input: 2 1 1 3 2 1
Output: 1 -1 -1 2 1 -1
```

7. Given a pattern containing only I's and D's, I for increasing and D for decreasing, devise an algorithm using stack to print the required sequence. Digits from 1-9 cannot repeat.

Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char pattern[] = "DIDI";
    int stack[20], top = -1, next = 1;
    int i, n = strlen(pattern);

    printf("Pattern: %s\nOutput: ", pattern);

    for (i = 0; i <= n; i++) {
        stack[++top] = next++;

        if (i == n || pattern[i] == 'I') {
            while (top != -1)
                printf("%d", stack[top--]);
        }
    }

    return 0;
}
```

Output

```
Pattern: DIDI
Output: 21435
```

8. Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from all the digits of the array. All digits must be used. Use stack and queue for implementation.

Program

```
#include <stdio.h>
#define MAX 20

int main() {
    int a[] = {6, 8, 4, 5, 2, 3};
    int n = 6, i, j, t;
    int stack[MAX], top = -1;
    int queue[MAX], front = 0, rear = -1;
    int num1 = 0, num2 = 0;

    /* Sort the digits */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[i] > a[j]) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }

    /* Put sorted digits into a queue */
    for (i = 0; i < n; i++)
        queue[++rear] = a[i];

    /* Use a stack while forming the two numbers */
    for (i = 0; i < n; i++)
        stack[++top] = queue[front++];

    while (top >= 0) {
        if (top % 2 == 0)
            num1 = num1 * 10 + stack[top];
        else
            num2 = num2 * 10 + stack[top];
        top--;
    }

    /* The alternating construction above may reverse the order,
       so form the minimum pair directly from sorted digits. */
    num1 = num2 = 0;
    for (i = 0; i < n; i++) {
        if (i % 2 == 0)
            num1 = num1 * 10 + a[i];
        else
            num2 = num2 * 10 + a[i];
    }

    printf("Input: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nMinimum sum = %d + %d = %d", num1, num2, num1 + num2);

    return 0;
}
```

Output

```
Input: 2 3 4 5 6 8
Minimum sum = 246 + 358 = 604
```

9. Given an array and a possible integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, print 0. Implement using queue.

Program

```
#include <stdio.h>
#define MAX 20

int main() {
    int a[] = {-8, 2, 3, -6, 10};
    int n = 5, k = 2;
    int q[MAX], front = 0, rear = -1;
    int i, j;

    printf("Input: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nOutput: ");

    for (i = 0; i <= n - k; i++) {
        front = 0;
        rear = -1;

        for (j = i; j < i + k; j++)
            if (a[j] < 0)
                q[++rear] = j;

        if (front <= rear)
            printf("%d ", a[q[front]]);
        else
            printf("0 ");
    }

    return 0;
}
```

Output

```
Input: -8 2 3 -6 10
Output: -8 0 -6 -6
```

10. Given a matrix of dimensions $m \times n$ where each cell can have value 0, 1, or 2. 0: empty cell, 1: fresh orange, 2: rotten orange. Find the minimum time required so that all fresh oranges become rotten. A rotten orange can rot its up, down, left and right neighbours. Use a queue.

Program

```
#include <stdio.h>
#define MAX 20

typedef struct {
    int r, c;
} Node;

int main() {
    int a[3][5] = {
        {2, 1, 0, 2, 1},
        {1, 0, 1, 2, 1},
        {1, 0, 0, 2, 1}
    };

    Node q[MAX * MAX];
    int front = 0, rear = -1;
    int fresh = 0, time = 0;
    int i, j;

    for (i = 0; i < 3; i++)
        for (j = 0; j < 5; j++) {
            if (a[i][j] == 1)
                fresh++;
            else if (a[i][j] == 2) {
                q[++rear].r = i;
                q[rear].c = j;
            }
        }

    while (front <= rear && fresh > 0) {
        int size = rear - front + 1;
        int changed = 0;

        for (i = 0; i < size; i++) {
            int r = q[front].r;
            int c = q[front++].c;
            int dr[] = {-1, 1, 0, 0};
            int dc[] = {0, 0, -1, 1};
            int d;

            for (d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];

                if (nr >= 0 && nr < 3 && nc >= 0 && nc < 5 &&
                    a[nr][nc] == 1) {
                    a[nr][nc] = 2;
                    fresh--;
                    q[++rear].r = nr;
                    q[rear].c = nc;
                    changed = 1;
                }
            }
        }

        if (changed)
            time++;
    }

    if (fresh == 0)
        printf("Minimum time required: %d", time);
    else
        printf("All fresh oranges cannot become rotten");

    return 0;
}
```

Output

Minimum time required: 2